

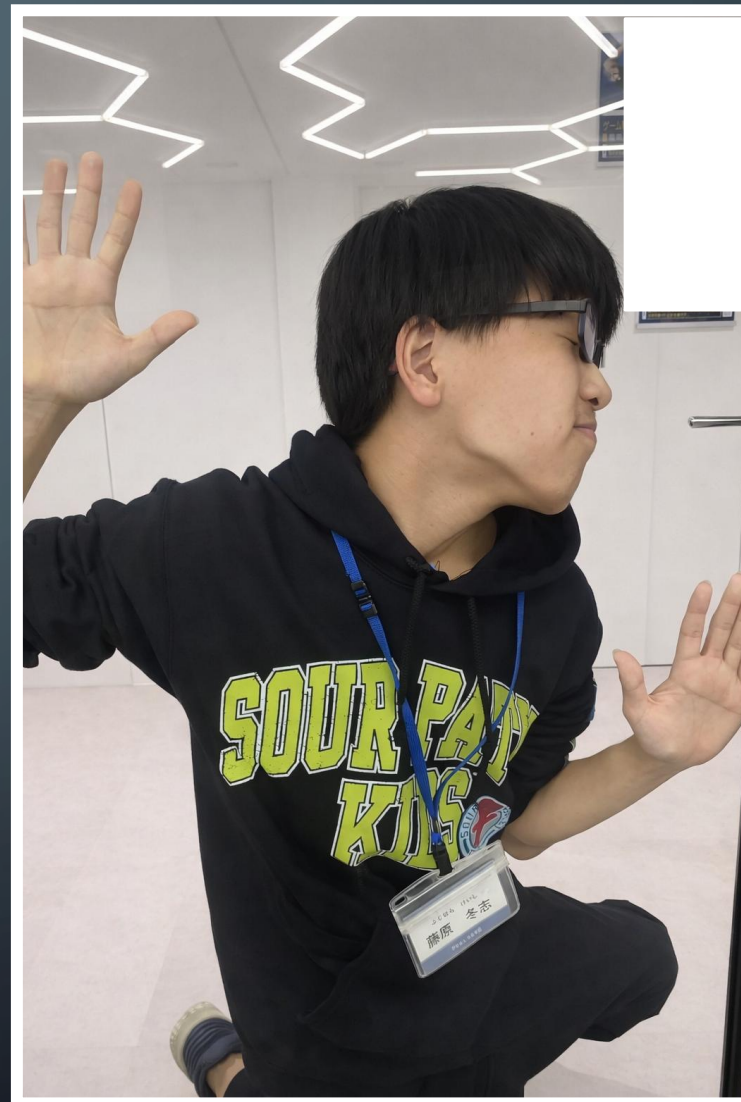
前のめりプログラマー

プロフィール

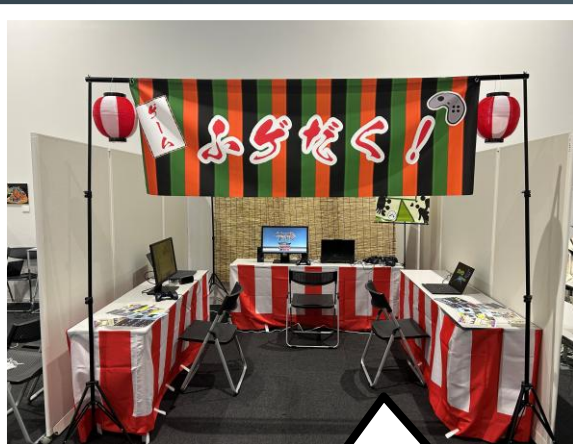
名前 : 藤原 冬志(ふじはら けいし)
学校 : 福岡情報ITクリエイター専門学校
学科 : ゲーム・クリエイター学科
メール : kf.syukatsu@gmail.com
GitHub : <https://github.com/keishiF>
YouTube : <https://www.youtube.com/@27-kf>

スキル

◆ C / C++	: 2年	◆ GitHub	: 2年
◆ C# / Unity	: 2年	◆ PowerPoint	: 3年
◆ Photoshop	: 2年	◆ Excel / Word	: 2年
◆ Illustrator	: 1年	◆ Blender	: 1年



自己PR



ふげだく! では制作だけでなく
イベントへの出展もしています!
九州学生ゲーム大祭2025では
得票数2位になりました!

ゲーム制作サークル「ふげだく!」
で副部長をしています!



休日はカラオケやボウリングに
行って楽しんでいます!

1	2	3	4	5	6	7	8	9	10	TOTAL
6	2	G	⑧	-	7	-	6	-	G	6
8	26	34	41	47	53	73	103	126	143	143



就活作品



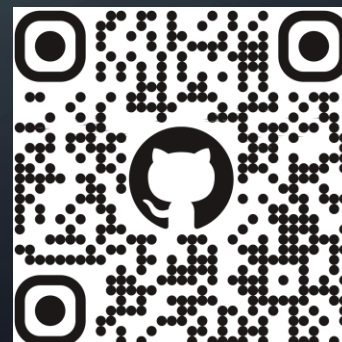
最終作品

DUNGEON HEARTS

作品名 : Dungeon Hearts
ジャンル : 3Dアクション
開発環境 : C++ / DxLib
制作人数 : 1人
制作期間 : 4か月
制作時期 : 2025年10月～2025年2月
担当箇所 : モデル、サウンド、一部エフェクト以外全て
対応機種 : Windows
GitHub : <https://github.com/keishiF/DungeonHearts.git>
動画URL : <https://youtu.be/2RGf7NZo9vg?si=yp05EDFA48XuklaT>



▼ GitHubQR

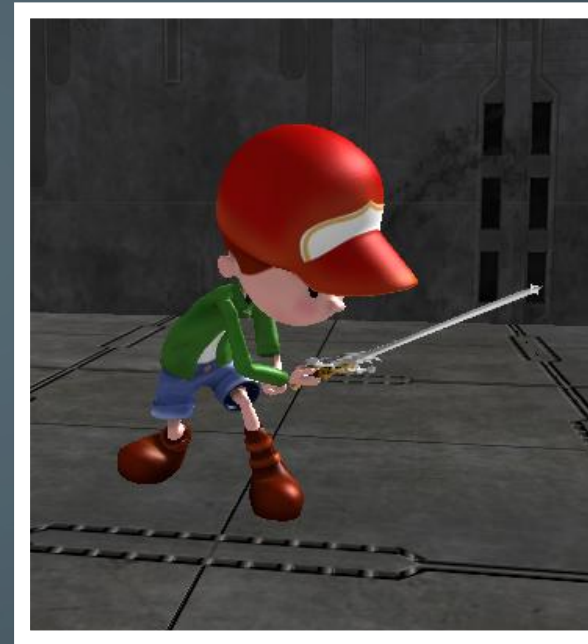
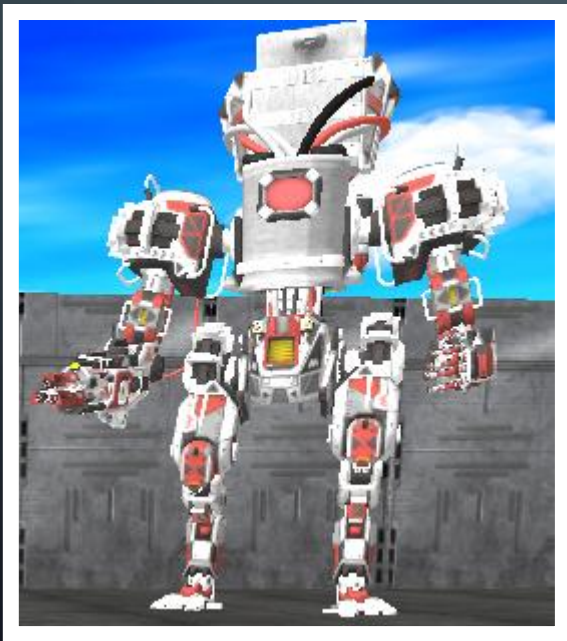


▼ YouTubeQR



ゲーム概要

- ◆地上攻撃、空中攻撃、遠距離攻撃
を駆使してステージを攻略せよ！
- ◆最奥にいるボスを倒せ！



DUNGEON HEARTS



技術紹介

場合に
consumeBuffer

lock () ->
make_share

ない場合は待機状

_m
lock () ->
make_share

ステート遷移時
virtual void OnEnter
新処理
virtual void
ステ
virtual void On

た場合は次の
consumeBuffered

_machine lock () -> ChangeStat
std::make_share PlayerSt

(カ
遷移

weak_ptr Machine<T>> machine.
machine = machine.
m_parent = parent.

時に一度呼ばれる処理
OnEntry () abstract:

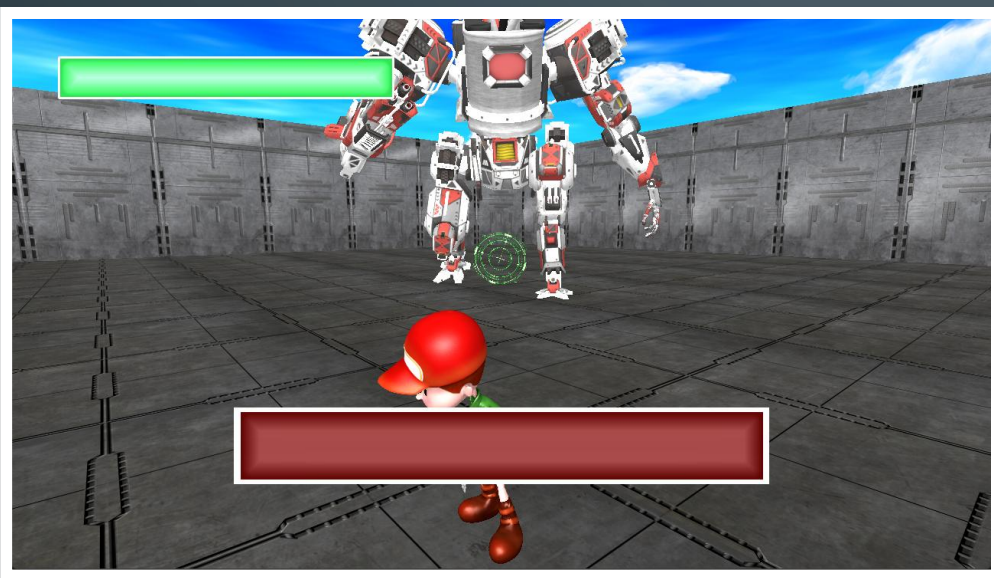
OnUpdate (std::shared_ptr<
ト離脱時に一度呼ばれる処理
virtual void OnLeave () abstract:

ected:
s
マシン
ne<l.

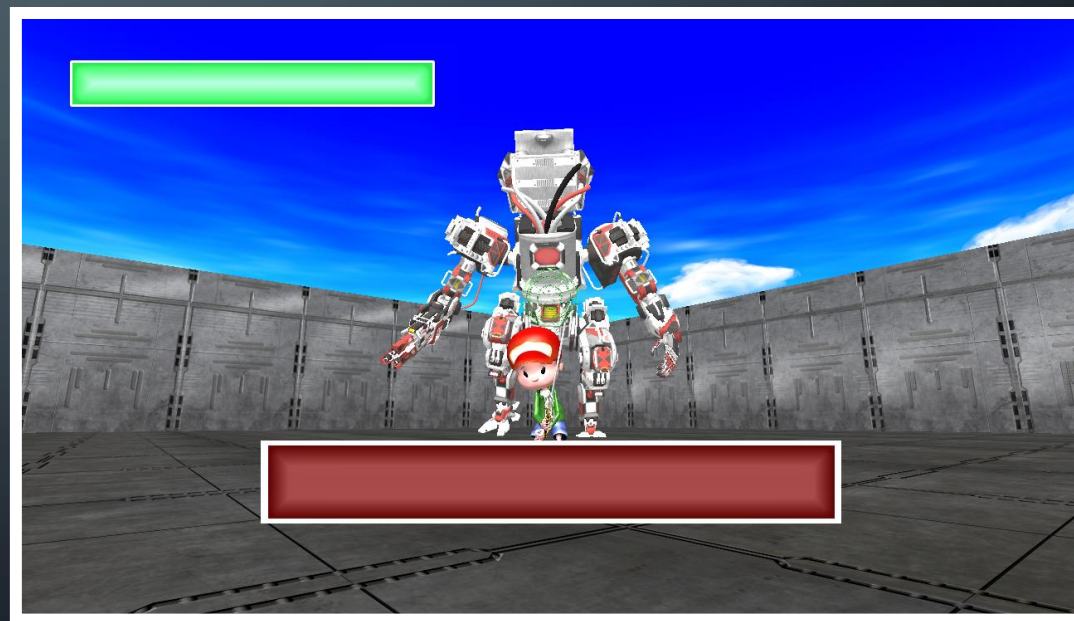
技術紹介① カメラワーク

ロックオン時の挙動を通常の敵とボスで分けました。
ボス専用カメラを作ることによって、敵に依存しない
視認性の高い快適な戦闘を実現しました。

▼ 専用カメラ実装前



▼ 専用カメラ実装後

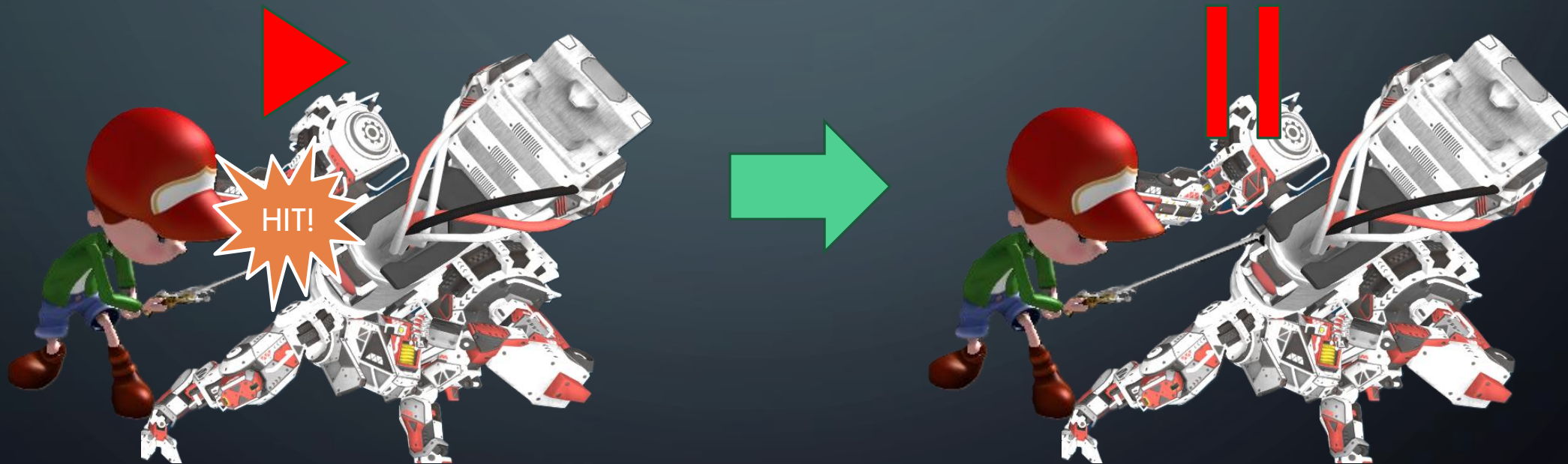


技術紹介② ヒットストップ

攻撃ヒット時に数フレーム更新を止めるヒットストップを実装しました。

HitStopManagerによって管理しています。

これにより攻撃した際に重みが生まれ、ゲーム体験が向上しました。



技術紹介③ 先行入力

入力を一定フレーム数保持する先行入力の仕組みを実装しました。

行動不可のタイミングに行われた入力を保持し、
遷移可能になったタイミングで処理することで入力の取りこぼしを防ぐことで、
プレイヤーの操作感を向上させました。

▼ 先行入力

```
bool Input::ConsumeBuffered(const char* key)
{
    // 指定されたキーが残っている場合は消費してtrueを返す
    if (!m_triggerBuffer.contains(key))
    {
        return false;
    }

    if (m_triggerBuffer[key] > 0)
    {
        m_triggerBuffer[key] = 0;
        return true;
    }

    return false;
}
```

▼ 状態遷移

```
// アニメーションの遷移フレームに達したら次の行動へ
if (currentFrame >= animTransFrame)
{
    // 入力があった場合は次の攻撃に遷移
    if (input.ConsumeBuffered("Attack"))
    {
        m_machine.lock()->ChangeState(
            std::make_shared<PlayerStateGroundAttack2>());
    }

    // 入力がない場合は待機状態に遷移
    else
    {
        m_machine.lock()->ChangeState(
            std::make_shared<PlayerStateIdle>());
    }
}

return;
```

技術紹介④ ステートマシン

テンプレートを用いることで、全てのキャラクターが利用できる
汎用的なステートマシンを実装しました。

キャラクターの状態ごとに個別クラスを実装し、入力や状況に応じて状態遷移を行います。

▼ ステートマシン

```
// T:ステートマシンを使うクラス
template<typename T>
class StateNode
{
public:
    // 初期化
    void Init(std::weak_ptr<StateMachine<T>> machine, std::weak_ptr<T> parent)
    {
        m_machine = machine;
        m_parent = parent;
    }

    // ステート遷移時に一度呼ばれる処理
    virtual void OnEntry () abstract;
    // 更新処理
    virtual void OnUpdate(std::shared_ptr<Camera> camera) abstract;
    // ステート離脱時に一度呼ばれる処理
    virtual void OnLeave () abstract;

protected:
    // ステートからステートマシン本体へアクセスするための弱参照
    std::weak_ptr<StateMachine<T>> m_machine;

    // ステートから親へアクセスするための弱参照
    std::weak_ptr<T> m_parent;
};
```

▼ 状態遷移

```
// ステート遷移
void ChangeState(std::shared_ptr<StateNode<T>> nextState)
{
    if (m_nowState)
    {
        m_nowState->OnLeave();
    }
    m_nowState = nextState;
    m_nowState->Init(this->weak_from_this(), m_parent);
    m_nowState->OnEntry();
}
```

▼ 使用例

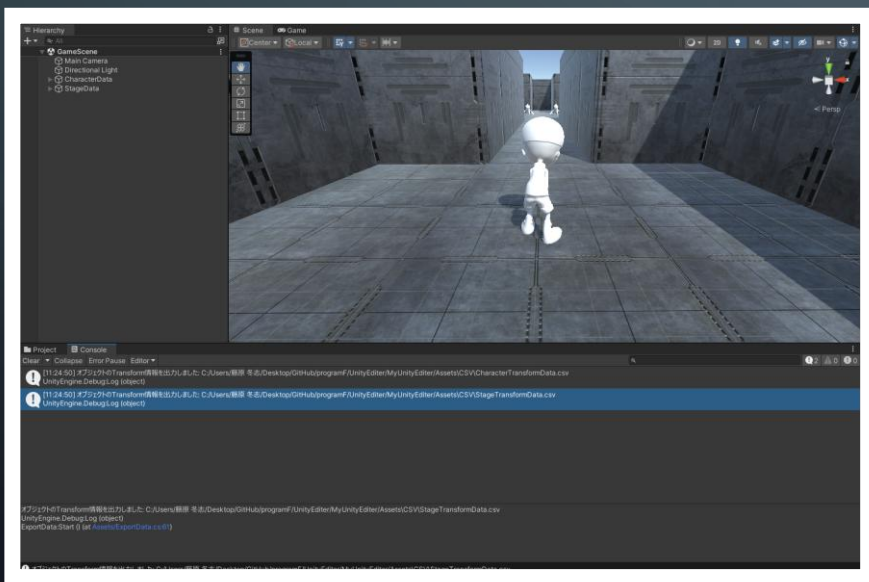
```
// ジャンプ入力で状態遷移
if (input.IsTrigger("Jump"))
{
    m_machine.lock()->ChangeState(
        std::make_shared<PlayerStateJump>());
    return;
}
```

```
// アニメーションが終了したら待機状態へ遷移
if (boss->GetAnimator().GetNextAnim().isEnd)
{
    m_machine.lock()->ChangeState(
        std::make_shared<BossStateIdle>());
    return;
}
```

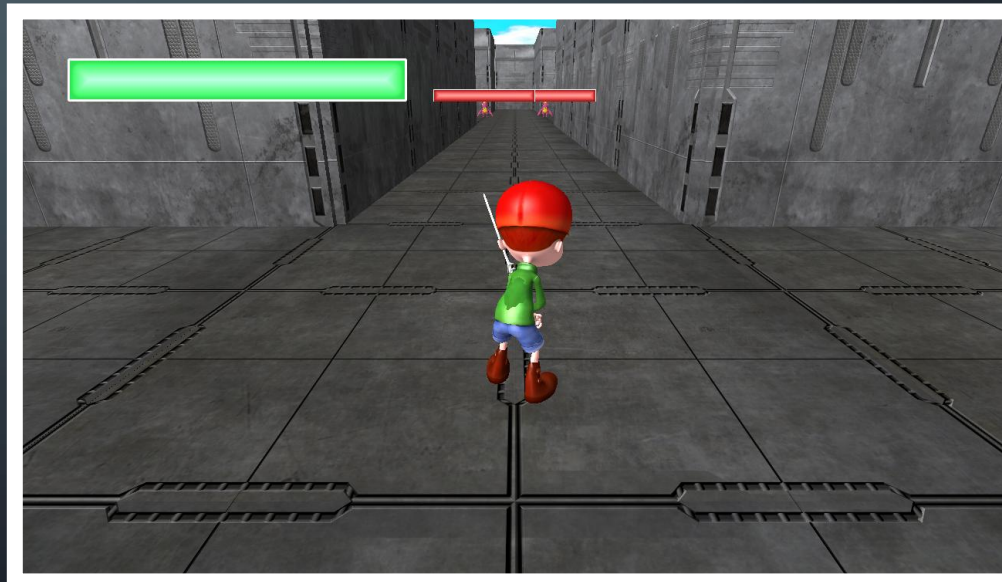
技術紹介⑤ ステージ制作

Unityをエディターとして使用しました。
Unity上で配置したオブジェクトのデータをCSVファイルに出力
それを読み込むことで、ステージとキャラの配置を簡単にできるようにしました。

▼ Unityで配置



▼ ゲーム内で読み込む



技術紹介⑥ 外部ファイル化

オブジェクトの配置情報やキャラクターのステータスを
外部ファイルで管理しています。
これにより、パラメータ調整を効率よく行えるようになりました。

▼ ステータス

Name	HP	Atk	MaxHP	RunSpeed
Player	25	1	25	15
EnemyMelee	1	2	1	5
EnemyFly	3	1	3	4
Boss	20	2	20	5

▼ キャラクターの配置情報

Name	PosX	PosY	PosZ	RotX	RotY	RotZ	ScaleX	ScaleY	ScaleZ	GroupID
Player	3	0	3	0	0	0	1	1	1	0
EnemyMe	6	0	55.5	0	0	0	1	1	1	1
EnemyMe	0	0	55.5	0	0	0	1	1	1	1
EnemyMe	-40	0	75	0	-90	0	1	1	1	2
EnemyMe	-40	0	87	0	-90	0	1	1	1	2
EnemyMe	-87	0	120	0	0	0	1	1	1	3
EnemyMe	-81	0	120	0	0	0	1	1	1	3
EnemyMe	-75	0	120	0	0	0	1	1	1	3
EnemyFly	-45	3	87	0	-90	0	1	1	1	2
EnemyFly	-45	3	75	0	-90	0	1	1	1	2
EnemyFly	-81	3	133	0	0	0	1	1	1	3
EnemyFly	-87	3	130	0	0	0	1	1	1	3
EnemyFly	-75	3	130	0	0	0	1	1	1	3
Boss	-81	0	201	0	0	0	1	1	1	0

▼ ステージの配置情報

Name	PosX	PosY	PosZ	RotX	RotY	RotZ	ScaleX	ScaleY	ScaleZ	GroupID
Floor	6	0	-6	0	0	0	1	1	1	0
Floor	0	0	-6	0	0	0	1	1	1	0
Floor	-6	0	-6	0	0	0	1	1	1	0
Floor	6	0	0	0	0	0	1	1	1	0
Floor	0	0	0	0	0	0	1	1	1	0
Floor	-6	0	0	0	0	0	1	1	1	0
Floor	6	0	6	0	0	0	1	1	1	0
Floor	0	0	6	0	0	0	1	1	1	0
Floor	-6	0	6	0	0	0	1	1	1	0
Floor	0	0	12	0	0	0	1	1	1	0
Floor	0	0	18	0	0	0	1	1	1	0
Floor	0	0	24	0	0	0	1	1	1	0
Floor	0	0	30	0	0	0	1	1	1	0
Floor	0	0	36	0	0	0	1	1	1	0
Floor	6	0	42	0	0	0	1	1	1	0
Floor	0	0	42	0	0	0	1	1	1	0
Floor	-6	0	42	0	0	0	1	1	1	0
Floor	6	0	48	0	0	0	1	1	1	0
Floor	0	0	48	0	0	0	1	1	1	0

作品総括

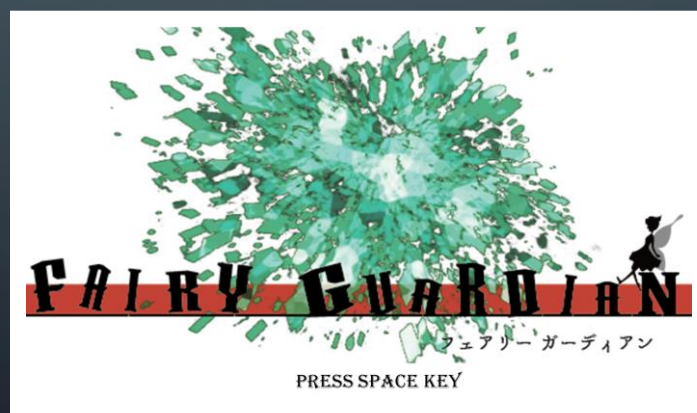
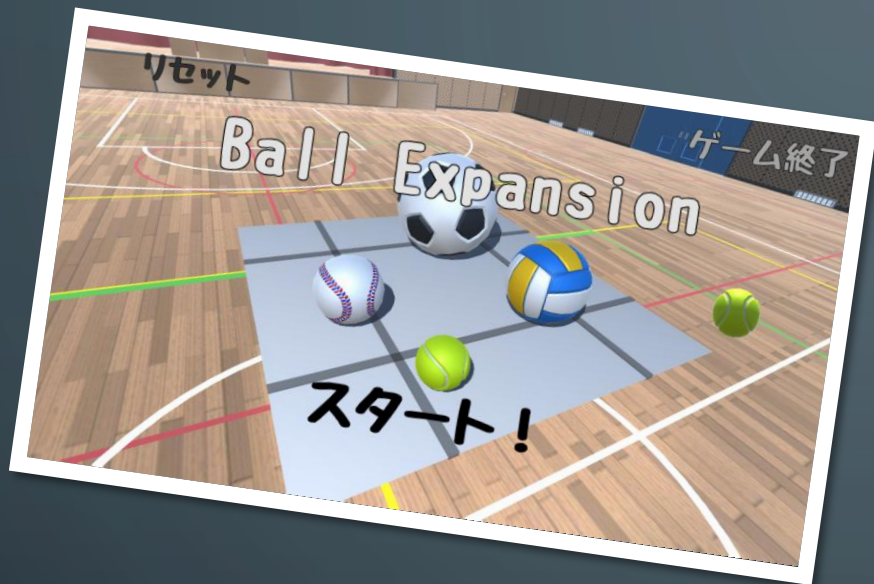
まだまだ**ボリュームが足りない！！**

上記の問題を解決するために

- ・ステージを増やす
- ・敵キャラやボスを追加

を行っていきます。

その他制作物



その他制作物 個人制作

作品名 : Knight Dungeon
ジャンル : 3Dアクション
開発環境 : Dxライブラリ / Visual Studio 2022
言語 : C++
制作時期 : 2025年6月～2025年8月
担当箇所 : モデル、サウンド以外全て

頑張ったこと・学んだこと

- ◆C++での3Dゲーム制作
- ◆クラス設計
- ◆Stateパターンを使った状態遷移
- ◆3D当たり判定



作品名 : 魔法使いの鏡
ジャンル : 2D縦スクロールアクション
開発環境 : Dxライブラリ / Visual Studio 2022
開発言語 : C++
制作時期 : 2024年11月～2025年1月
担当箇所 : リザルトシーン

頑張ったこと・学んだこと

- ◆Dxライブラリを使用したゲーム制作
- ◆2D当たり判定

その他制作物 チーム制作①

作品名 : Ball Expansion
ジャンル : 3Dパズル
開発環境 : C# / Unity
制作人数 : プログラマー3人
制作時期 : 2024年9月～2024年10月
担当箇所 : ボールを落として
ボール同士が合体する処理

頑張ったこと・学んだこと

- ◆ 短期間制作でのスケジューリングと役割分担
- ◆ ボール同士が合体する処理

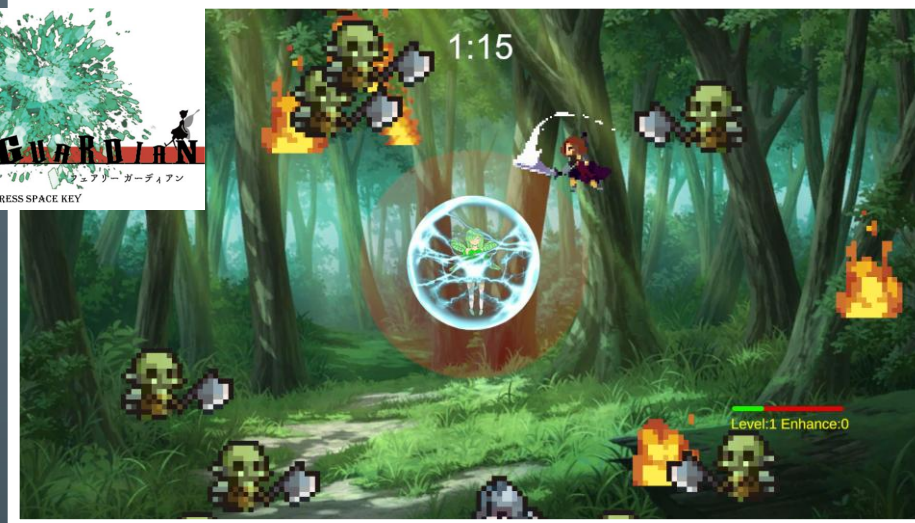


その他制作物 チーム制作②

作品名 : FAIRY GURDIAN
ジャンル : 2Dタワーディフェンス
開発環境 : Unity
言語 : C#
制作時期 : 2024年5月～2025年7月
担当箇所 : バリアの処理、プログラムリーダー

頑張ったこと・学んだこと

- ◆初めてのチーム開発経験
- ◆Gitの使い方



作品名 : ワンニャンデカシンカ
ジャンル : 2D早打ちバトル
開発環境 : Unity
開発言語 : C#
制作時期 : 2025年10月4日～2025年10月5日
担当箇所 : リザルトシーン

頑張ったこと・学んだこと

- ◆ゲームジャムでの制作経験
- ◆他職種の方との連携

今後の目標

キャラクター挙動を担うプログラマーになる。

これまでの制作で、キャラクター挙動を制作しているときが1番楽しかった。
そしてなにより、最もゲームの遊びやすさ・楽しさに繋がっていると知った。
なのでこれからユーザーがストレスを感じない
より良いキャラクター挙動を実現するために
物理や数学的知識を学習していきたいと思います。